Estudo Dirigido - TAD Pilha

Algoritmos e Estrutura de Dados - PUC Minas Contagem - 2026/2

Aluno: Romulo Correa Matrícula: 1596721

Parte 2 - Rastreio no papel (Pilha com capacidade 4)

Operação	Retorno	Pilha (base -> topo)	topo
empilha(10)	-	[10]	1
empilha(20)	-	[10, 20]	2
desempilha()	20	[10]	1
empilha(30)	-	[10, 30]	2
empilha(40)	-	[10, 30, 40]	3
desempilha()	40	[10, 30]	2
desempilha()	30	[10]	1
empilha(50)	-	[10, 50]	2

Pare e pense. Ao final, o vetor interno é **[10, 50, 40, 0]**. Portanto **item[2] = 40**. Esse valor **não** faz parte da pilha: com topo = 2, a pilha são apenas as posições 0 e 1; a posição 2 é a próxima posição livre, e o 40 é apenas um resíduo (lixo) da operação anterior. O desempilha não precisa apagar nada porque o que define quais elementos existem é o valor de **topo**, não o conteúdo do vetor: basta recuar o topo em uma unidade. A posição "liberada" será simplesmente sobrescrita no próximo empilha. Isso torna as duas operações O(1), sem mover elementos.

Parte 4.1 - Rastreio do Bubble Sort com pilhas para {5, 2, 9, 1, 7}

Estado inicial: A (base -> topo) = [5, 2, 9, 1, 7]; B e R vazias.

Passada	carry final	B (base -> topo)	R (base -> topo)
1	9	[1, 7, 2, 5]	[9]
2	7	[2, 5, 1]	[9, 7]
3	5	[1, 2]	[9, 7, 5]
4	2	[1]	[9, 7, 5, 2]
5	1	[ ] (vazia)	[9, 7, 5, 2, 1]

Ao desempilhar R, saem 1, 2, 5, 7, 9: vetor em ordem crescente.

Parte 4.3 - Questões

(a) Para n = 5 foram feitas **10 comparações** (4 + 3 + 2 + 1 + 0, pois a passada com 'restantes' elementos faz restantes - 1 comparações). A expressão geral é (n-1) + (n-2) + ... + 1 + 0 = **n(n-1)/2**, ou seja, $O(n^2)$. O Bubble Sort clássico sem otimização (sem parada antecipada) faz exatamente o mesmo número: n(n-1)/2. A versão com pilhas não perde nem ganha em comparações; também não tem parada antecipada, então um vetor já ordenado continua com 10 comparações para n = 5.

(b) O Bubble Sort com vetor ordena no próprio vetor, usando memória extra O(1) (uma variável auxiliar para a troca). A versão com pilhas aloca três pilhas (A, B e R), cada uma com capacidade n, ou seja, cerca de **3n posições** a mais, além da variável carry: custo extra **O(n)** de memória.

(c) Deixamos de ter o **acesso direto por índice** (v[i]) e, com ele, a troca de posições (v[i] <-> v[i+1]). O código compensa de duas formas: (1) a variável **carry** carrega a "bolha" e faz o papel do elemento que seria trocado de lugar; (2) os elementos são **transferidos entre pilhas** (A -> B), um por vez, de modo que cada passada percorre todos os elementos restantes, e os papéis de A e B são trocados ao fim da passada. O maior de cada passada vai para R.

(d) Em geral **não vale a pena** ordenar com pilhas: o algoritmo continua $O(n^2)$, gasta O(n) de memória extra e é mais trabalhoso que ordenar o vetor diretamente (ou usar um algoritmo O(n log n)). A ideia faz sentido quando só se pode acessar os dados pelo topo, como em uma estrutura LIFO que não oferece acesso por índice (por exemplo, uma pilha de tarefas ou de desfazer/refazer que precise ser reordenada) ou em exercícios que testam o domínio do TAD. Nesses casos a ideia de mover os elementos entre pilhas com uma variável auxiliar é útil.

Desafio (opcional) - ordenaPilha

Implementado em Pilha.cs. Resultado do teste: entrada [5, 2, 9, 1, 7] -> saída [1, 2, 5, 7, 9] (maior valor no topo).